

Introduction to the use of PKA key wrapping with coupling and chaining bridge for STM32 MCUs

Introduction

This application note introduces the use of PKA key wrapping with the coupling and chaining bridge (CCB) peripheral available in a selection of STM32 microcontrollers (MCUs). The MCU is listed in [Table 1](#).

The document is structured as follows:

- Why do we need PKA key wrapping? See [Section 1](#).
- What are the cryptographic functions involved in the wrapping of PKA keys? See [Section 2](#).
- What set of functions does STM32Cube HAL provide to run the updated cryptographic functions? See [Section 3](#).

Table 1. Applicable products

Type	Products
Microcontroller	STM32C59x/C5A3 line
	• STM32H553CG, STM32H553RG, STM32H553UG, STM32H553VG, STM32H553ZG • STM32H5F4AJ, STM32H5F4IJ, STM32H5F4VJ, STM32H5F4ZJ • STM32H5F5IJ, STM32H5F5LJ, STM32H5F5VJ, STM32H5F5ZJ
	• STM32U385CG, STM32U385KG, STM32U385RC, STM32U385RE, STM32U385RG, STM32U385VG • STM32U3C5CI, STM32U3C5JI, STM32U3C5QI, STM32U3C5RI, STM32U3C5VI, STM32U3C5WI, STM32U3C5ZI



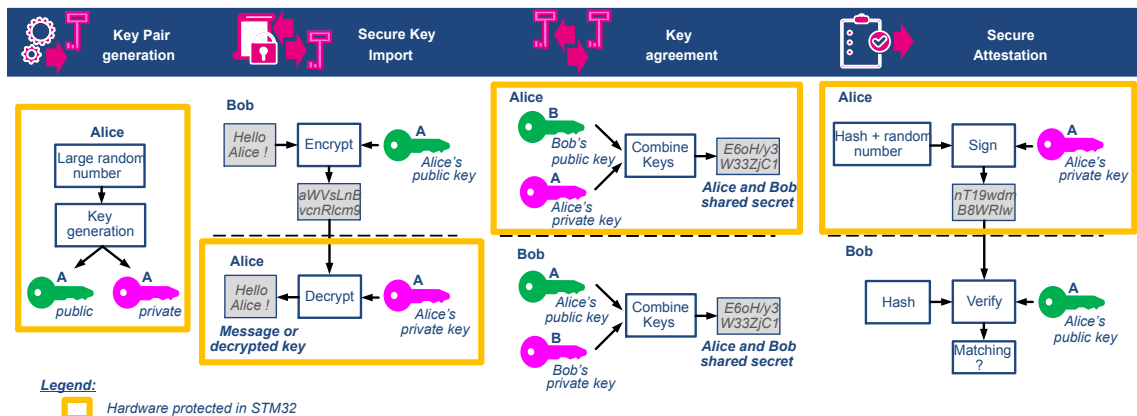
1 Why do we need PKA key wrapping?

This section concerns the use of private keys and why it is helpful to wrap them.

1.1 Private key usage

Figure 1 summarizes the possible usage of private keys in the PKA peripheral. This usage is highlighted in yellow.

Figure 1. Cryptographic usages for private keys



DT77016V1

Private keys are used in secure key import, a crucial process in cryptographic systems that ensures that cryptographic keys are transferred securely from one entity to another without being exposed to unauthorized parties. Figure 1 shows a secure transfer of data from Bob to Alice using the RSA method. The only prerequisite for this transfer is for Bob to hold the genuine public key of Alice.

A more complex way of sharing an encryption key between two parties is called cryptographic key agreements. Those methods are protocols that allow two or more parties to securely establish a shared secret key over an insecure communication channel. This shared secret key can then be used for subsequent encryption and decryption of messages between the parties. Figure 1 shows a key agreement between Bob and Alice, with both parties holding the genuine public key of the other party.

Finally, private keys are also used for secure attestation. It is a process by which a device or system proves its integrity and authenticity to another party. At the end of the secure attestation process the other party accepts (or not) to interact with this device or system, because it got the proof that the device is running trusted software and has not been tampered with. This method is crucial in various applications, including secure boot processes, trusted computing, and IoT security. Figure 1 shows the principle of a secure attestation using the ECC method.

Note: For all those use case elliptic curves or RSA/DSA variants exists.

1.2 Private key protection

Private keys are created during a process called digital key pair generation, shown in Figure 1. It is a fundamental process in cryptography, during which two mathematically related keys are created: a public key and a private key.

The security of asymmetric cryptography relies heavily on the protection of the private key. For example, as the private key is used to decrypt messages that were encrypted with the user's public key, if someone gains access to a private key, it is then possible to decrypt and read confidential messages.

Also, as the private key is used to create digital signatures that verify the user's identity, if someone other than the user has access to the private key, the user's digital signature can be forged, making it appear as though the user has approved or sent messages that were not written by the user.

Lastly, when the private key ensures that the messages sent by the user are not tampered with, if someone other than the user has the private key, the messages can be altered and still make them appear as if they are from the user.

1.3 Private key wrapping in STM32

As described before, the private key usage can be critical for device security. Hence it is important to make it as secure as possible. On the STM32 microcontroller listed in [Table 1](#), wrapped private keys can be stored anywhere, because they can only be used on this device. Also, the software that manipulates wrapped private keys can never access its value in the clear, making the overall application more robust against attacks.

This optional hardware protection relies on a CCB peripheral that enforces the encryption (or wrapping) of PKA private keys using a 256-bit AES key loaded in side-channel protected SAES engine.

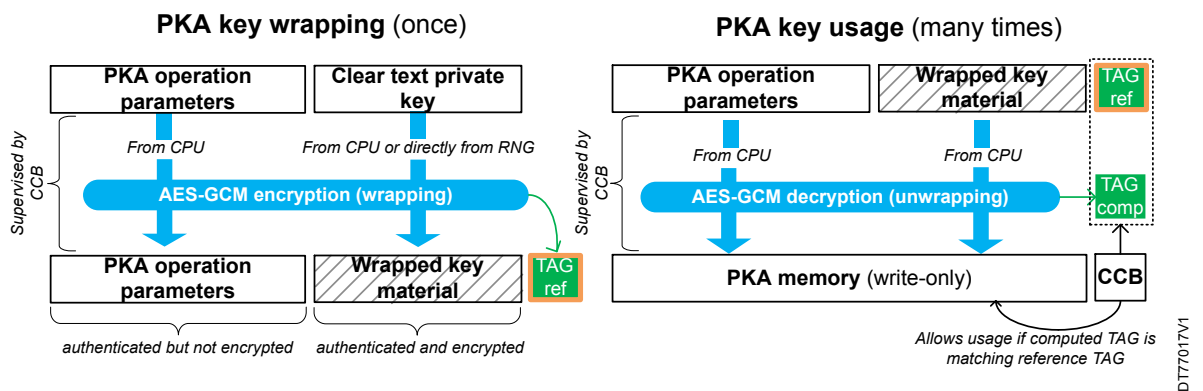
2 Cryptographic primitives with PKA key wrapping

This section describes the principle of PKA private keys wrapping using AES. It also summarizes the PKA operations involved in the use cases described in [Section 1.1: Private key usage](#).

2.1 AES: AES-GCM authenticated encryption with associated data (AEAD)

AES-GCM (Advanced Encryption Standard - Galois/Counter Mode) is a widely used cryptographic method that provides both confidentiality and data integrity. It combines the AES encryption algorithm with the Galois/Counter Mode of operation to achieve authenticated encryption. The PKA key wrapping with coupling and chaining bridge (CCB) uses the AES-GCM algorithm, as shown in [Figure 2](#). The key size used is 256-bit.

Figure 2. AES-GCM usage for PKA key wrapping



Under the supervision of the CCB, the *PKA operation parameters* are only authenticated while the *key material* is authenticated and encrypted using AES-GCM. At the end of the wrapping (or encryption) process, the application must store the *reference tag* together with the operation parameters and the wrapped key material. Indeed, once the PKA wrapped key is decrypted in PKA memory, PKA can use it with the verified operation parameters only if the reference and the computed AES-GCM tags match.

2.2 AES: Key wrapping in SAES peripheral

For PKA key wrapping or unwrapping the application must load a key in SAES peripheral before performing the required AES-GCM operation shown in [Figure 2](#). To ensure the wrapped key material is never seen in the clear, the application must select the DHUK (with optional XOR with BHK) or unwrap in the SAES an existing PKA key wrapping encryption key that has been encrypted in wrapped-key mode using the DHUK (with optional XOR with BHK). STM32 HAL exports application programming interface (API) functions that help manage this scenario, as described in [Section 3.1.2](#), [Section 3.2.2](#), and [Section 3.3.2](#).

2.3 PKA: Scalar multiplication

STM32Cube HAL proposes the scalar multiplication method to implement the following use cases, listed in [Section 1.1](#): ECC key generation, ECDH key agreement and ECC secure key import.

When an application needs a given ECC public key, it can use the related private key with the STM32Cube HAL scalar multiplication API, adding the ECC curve base point as a parameter.

Also, ECDH key agreement requires the application to use the scalar multiplication API with the related private key and the other party public key as point on the ECC curve. The agreed AES key is usually the result of a KDF2 using the X coordinate of the previously calculated scalar multiplication result. This last part is out of scope of the application note.

Lastly, ECC secure key import requires the application to use the scalar multiplication API with the other party public key as a point on the ECC curve. Application then compute a KDF with the X coordinate of the point previously computed, using the result as an AES key to decrypt the key to import. This last part is out of scope of this application note.

In a legacy application, STM32 HAL provides the scalar multiplication API function below:

```
HAL_StatusTypeDef HAL_PKA_ECCMul(PKA_HandleTypeDef *hpka,
    PKA_ECCMulInTypeDef *in,
    uint32_t Timeout);

typedef struct
{
    uint32_t scalarMulSize;           /*!< Number of element in scalarMul array */
    uint32_t modulusSize;            /*!< Number of element in modulus, coefA,
                                     pointX and pointY arrays */
    uint32_t coefSign;               /*!< Curve coefficient a sign */
    const uint8_t *coefA;            /*!< Pointer to curve coefficient |a| */
    const uint8_t *coefB;            /*!< pointer to curve coefficient b */
    const uint8_t *modulus;          /*!< Pointer to curve modulus value p */
    const uint8_t *pointX;           /*!< Pointer to point P coordinate xP */
    const uint8_t *pointY;           /*!< Pointer to point P coordinate yP */
    const uint8_t *scalarMul;        /*!< Pointer to scalar multiplier k */
    const uint8_t *primeOrder;       /*!< pointer to order of the curve */
} PKA_ECCMulInTypeDef;
```

STM32 HAL also provides the scalar multiplication API function with PKA wrapped keys, as described in [Section 3.2](#).

2.4 PKA: ECDSA signature

STM32Cube HAL proposes the ECDSA signature method to implement the ECC secure attestation, listed in [Section 1.1](#). For this use case the application must do the following:

1. Select an ECC curve and generates a private key using the TRNG (size depends on the selected curves).
2. Compute the hash of the message for which a signature is needed.
3. Use the STM32 HAL ECDSA signature API to compute the signature (r, s). A random nonce must also be provided, unless the CCB is used (see [Section 3.1](#)).
4. When distributing the signature as an attestation, make sure the corresponding public key is available to the remote verifier. This public key can be computed using the scalar multiplication API and the ECC curve base point G (see [Section 2.3](#)).

In a legacy application STM32 HAL provides the ECDSA signature function below.

```
HAL_StatusTypeDef HAL_PKA_ECDSASign(PKA_HandleTypeDef *hpka,
    PKA_ECDSASignInTypeDef *in,
    uint32_t Timeout);

typedef struct
{
    uint32_t primeOrderSize;         /*!< Number of element in primeOrder array */
    uint32_t modulusSize;           /*!< Number of element in modulus array */
    uint32_t coefSign;              /*!< Curve coefficient a sign */
    const uint8_t *coef;            /*!< Pointer to curve coefficient |a| */
    const uint8_t *coefB;           /*!< Pointer to B coefficient */
    const uint8_t *modulus;         /*!< Pointer to curve modulus value p */
    const uint8_t *integer;         /*!< Pointer to random integer k */
    const uint8_t *basePointX;      /*!< Pointer to curve base point xG */
    const uint8_t *basePointY;     /*!< Pointer to curve base point yG */
    const uint8_t *hash;            /*!< Pointer to hash of the message */
    const uint8_t *privateKey;      /*!< Pointer to private key d */
    const uint8_t *primeOrder;      /*!< Pointer to order of the curve n */
} PKA_ECDSASignInTypeDef;

typedef struct
{
    uint8_t *RSign;                 /*!< Pointer to signature part r */
    uint8_t *SSign;                 /*!< Pointer to signature part s */
} PKA_ECDSASignOutTypeDef;
```

STM32 HAL also provides the ECDSA signature function with PKA wrapped keys, as described in [Section 3.1](#).

2.5 PKA: Modular exponentiation

STM32Cube HAL proposes the modular exponentiation method to implement the following use cases, listed in [Section 1.1: Private key usage](#): RSA key agreement, RSA secure key import and RSA attestation. Details on this are out of scope of this application note.

In a legacy application, STM32 HAL provides the modular exponentiation function below.

```
HAL_StatusTypeDef HAL_PKA_ModExpProtectMode(PKA_HandleTypeDef *hpka,
      PKA_ModExpProtectModeInTypeDef *in,
      uint32_t Timeout);

typedef struct
{
    uint32_t expSize;           /*!< Size of the operand in bytes */
    uint32_t OpSize;           /*!< Size of the operand in bytes */
    const uint8_t *pOp1;       /*!< Pointer to Operand 1 */
    const uint8_t *pExp;       /*!< Pointer to Exponent */
    const uint8_t *pMod;       /*!< Pointer to modulus */
    const uint8_t *pPhi;       /*!< Pointer to Phi value */
} PKA_ModExpProtectModeInTypeDef;
```

STM32 HAL also provides the modular exponentiation function with PKA wrapped keys, as described in [Section 3.3](#).

3 Wrapped key usage with the CCB

This section details how private keys in their wrapped form can be used in the STM32 microcontrollers listed in Table 2.

Table 2. Definition of product types

Products		
Type 1 (PT1)	Type 2 (PT2)	Type 3 (PT3)
STM32H5F4xx, STM32H5F5xx STM32U385xx	STM32H553xx STM32U3C5xx	STM32C59x/C5A3

3.1 ECDSA signature

STM32Cube HAL proposes a set of functions to compute ECDSA signatures with PKA wrapped keys.

3.1.1 Key generation

ECC private key is generated with the RNG. In FIPS PUB 186-4 specification “*Digital Signature Standard (DSS)*” section B.4 NIST proposes two methods for the generation of the ECC private key (*extra random bits* or *testing candidates*). Application must select one of those two methods when computing private keys for ECDSA signature generation.

Public key computation is described in Step 3 of Section 3.1.3 for product type 1 (PT1), in step 3 of Section 3.1.4 for product type 2 (PT2), and in Step 3 of Section 3.1.5 for product type 3 (PT3).

3.1.2 Blob creation, blob usage

Wrapping a private key for ECDSA signatures corresponds to creating a ECDSA signature key blob. Its usage is summarized in Table 3.

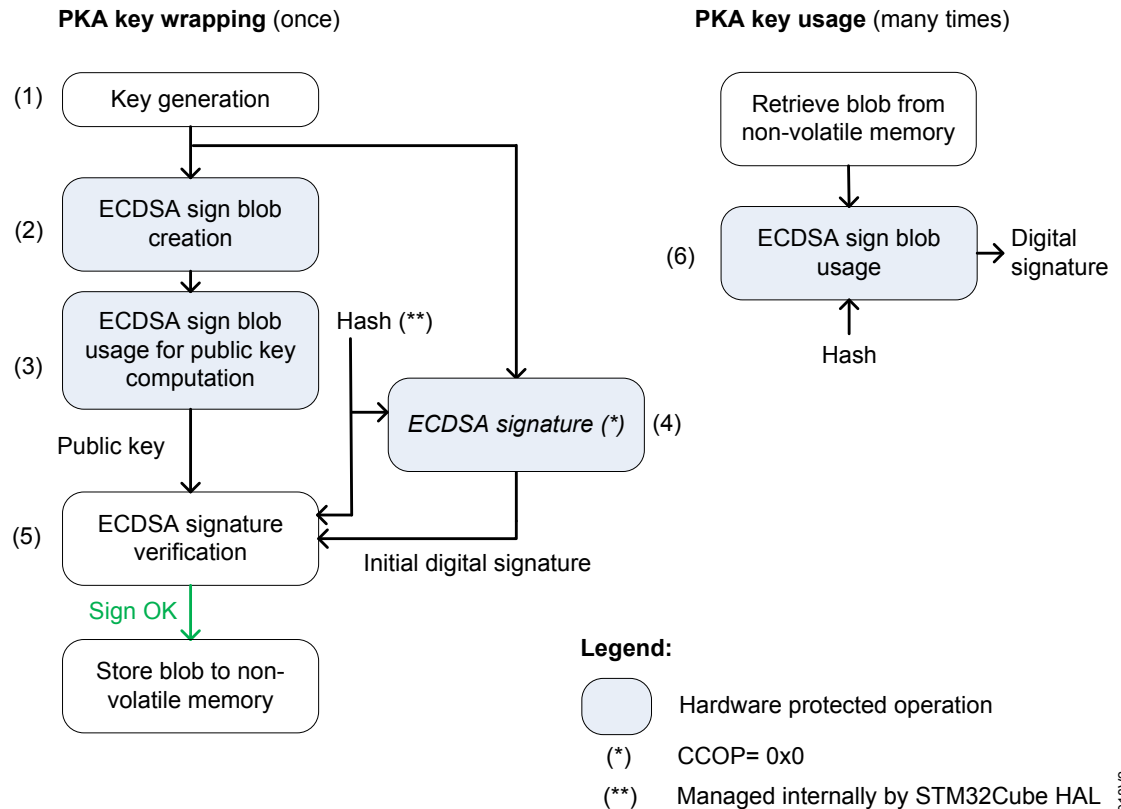
Table 3. ECDSA signature key blob usage

Attestation	Key import	Key agreement	Key generation	Usage in PKA
X	-	-	X	ECDSA signature Scalar multiplication

ECDSA signature blob creation and usage methods are summarized in Section 3.1.3 for product type 1 (PT1), in Section 3.1.4 for product type 2 (PT2), and in Section 3.1.5 for product type 3 (PT3).

3.1.3 Blob creation, blob usage (PT1 code)

Figure 3. ECDSA signature blob creation and usage (PT1)



DT77018V2

STM32Cube HAL supports the following methods to implement the flow in Figure 3. These methods apply to product type 1 defined in Table 2:

1. Generate keys.
2. Protected ECDSA signature blob creation, with a selected ECC curve and the generated private key as input.
3. Protected scalar multiplication, with ECC curve base point G and previously computed blob as input. Result is the computed ECDSA signature public key.
4. ECDSA signature using the generated private key as input. Any hash information can be used. Call the result of this signature (r, s).
5. ECDSA signature verification, using the reference hash information, the signature (r, s) and the computed ECDSA signature public key. If the API returns that the signature is correct, save the ECDSA signature blob created in step 2.
6. ECDSA key blob can be used at any time to compute ECDSA signature using a wrapped key.

Step 1 code:

Generate private key `ClearPrivateKey` with the TRNG, as defined in Section 3.1.1.

Step 2, 3, 4 and 5 code:

If the user wants to wrap `ClearPrivateKey` with 256-bit DHUK (or DHUK XOR BHK), select `WrappingKeyType = HAL_CCB_USER_KEY_HW (or HAL_CCB_USER_KEY_HSW)`.

If the user wants to wrap `ClearPrivateKey` with a known `ClearAESKey` this user-defined wrapping key must first be encrypted by calling the `HAL_CCB` function below, with `WrappingKeyType = HAL_CCB_USER_KEY_WRAPPED`.

The recommended algorithm is AES-CBC with 256-bit DHUK, selecting `AES_Algorithm = CCB_AES_CBC` and initializing `InitVect`.

```
HAL_StatusTypeDef HAL_CCB_ECDSA_WrapSymmetricKey(CCB_HandleTypeDef *hccb,
  const uint32_t *pClearAESKey,
  CCB_WrappingKeyTypeDef *pWrappingKey);

typedef struct
{
  uint32_t WrappingKeyType;          /*!< This parameter can be value of
                                     @ref CCB_WrappingKeyTypeDef          */
  uint32_t AES_Algorithm;            /*!< Used only when wrappingkey type is
                                     HAL_CCB_USER_KEY_WRAPPED, AES Algorithm.
                                     This parameter can be a value of
                                     @ref CCB_AES_Algorithm_Mode          */
  uint32_t *pInitVect;              /*!< Used only when wrappingkey type is
                                     HAL_CCB_USER_KEY_WRAPPED, Pointer to
                                     the initialization vector, counter with
                                     CBC Algo                               */
  uint32_t *pUserWrappedWrappingKey; /*!< Used only when wrappingkey type is
                                     HAL_CCB_USER_KEY_WRAPPED, Pointer to the
                                     wrapped wrapping Key                    */
} CCB_WrappingKeyTypeDef;
```

Wrap the `ClearPrivateKey` with the `HAL_CCB` function below. Keep the `WrappedPrivateKeyBlob` result in volatile memory.

```
HAL_StatusTypeDef HAL_CCB_ECDSA_WrapPrivateKey(CCB_HandleTypeDef *hccb,
  CCB_ECDSACurveParamTypeDef *pCurveParam,
  uint8_t *pClearPrivateKey,
  CCB_WrappingKeyTypeDef *pWrappingKey,
  const CCB_ECDSAKeyBlobTypeDef *pWrappedPrivateKeyBlob);
```

Above function also do the following:

- Generate the public key, as defined in Step 3 (local/ internal variable).
- Generate the ECDSA signature, as defined in Step 4 (local/ internal variable).

Lastly, as defined in Step 5, `HAL_CCB_ECDSA_WrapPrivateKey` verifies the generated signature with the generated public key. If the signature verification returns true `WrappedPrivateKeyBlob` can be saved anywhere in non-volatile memory.

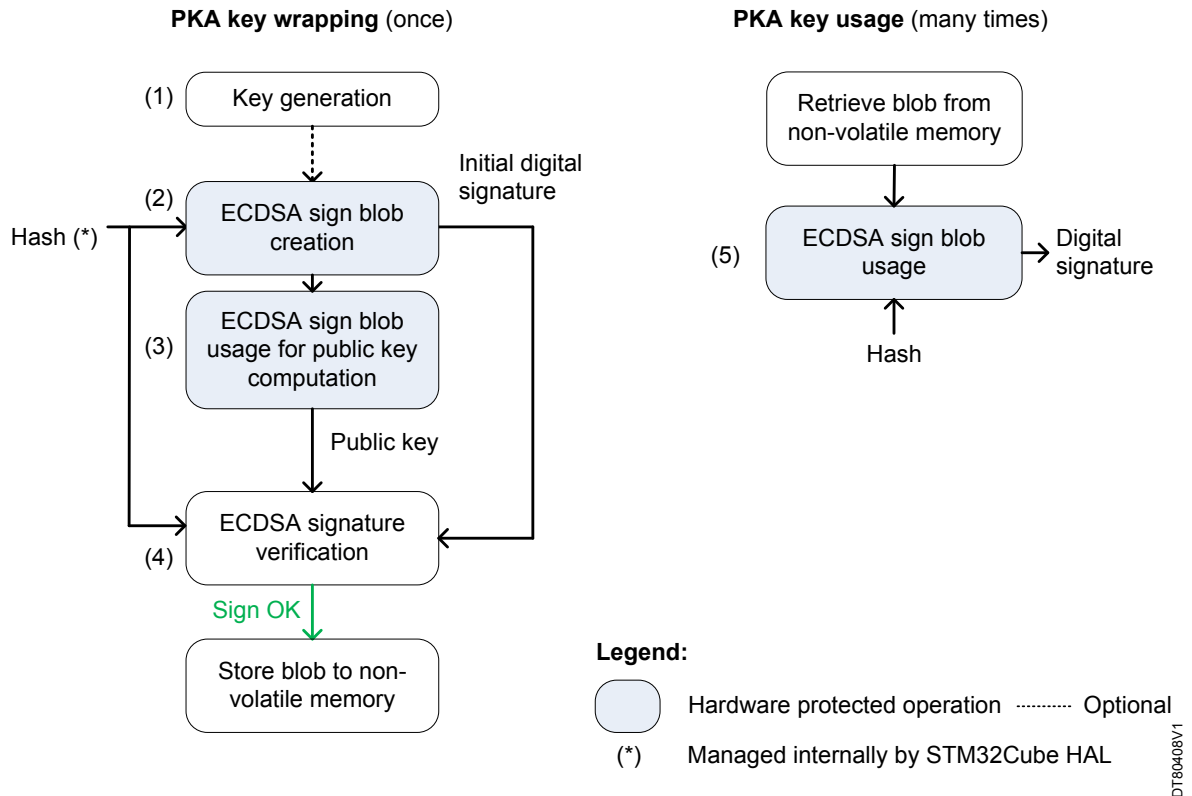
Step 6 code:

At any time, a signature can be computed using `WrappedPrivateKeyBlob` and the `HAL_CCB` function:

```
HAL_StatusTypeDef HAL_CCB_ECDSA_Sign(CCB_HandleTypeDef *hccb,
  CCB_ECDSACurveParamTypeDef *pCurveParam,
  CCB_WrappingKeyTypeDef *pWrappingKey,
  CCB_ECDSAKeyBlobTypeDef *pWrappedPrivateKeyBlob,
  uint8_t *pHash,
  CCB_ECDSASignTypeDef *pSignature);
```

3.1.4 Blob creation, blob usage (PT2 code)

Figure 4. ECDSA signature blob creation and usage (PT2)



STM32Cube HAL supports the following methods to implement the flow in Figure 4. These methods apply to product type 2 defined in Table 2:

1. Generate private key (optional).
2. Protected ECDSA signature blob creation, with a selected ECC curve and any hash information. Use the step 1 private key as input, or allow the device to generate a key in a confidential way. The result includes the initial signature (r, s).
3. Protected scalar multiplication, with ECC curve base point G and previously computed blob as input. Result is the computed ECDSA signature public key.
4. ECDSA signature verification uses the reference hash information, the initial signature (r, s), and the computed ECDSA signature public key. If the API returns that the signature is correct, save the ECDSA signature blob created in step 2.
5. ECDSA key blob can be used at any time to compute ECDSA signature using a wrapped key.

Step 1 code:

If the key is generated by application, refer to the Step 1 code in Section 3.1.3.

Step 2, 3 and 4 code:

If the user wants to wrap `ClearPrivateKey` with 256-bit DHUK (or DHUK XOR BHK), select `WrappingKeyType = HAL_CCB_USER_KEY_HW` (or `HAL_CCB_USER_KEY_HSW`).

If the user wants to wrap `ClearPrivateKey` with a known `ClearAESKey` this user-defined wrapping key must first be encrypted by calling the `HAL_CCB` function below, with `WrappingKeyType = HAL_CCB_USER_KEY_WRAPPED`.

The recommended algorithm is AES-CBC with 256-bit DHUK, selecting `AES_Algorithm = CCB_AES_CBC` and initializing `InitVect`.

```

HAL_StatusTypeDef HAL_CCB_ECDSA_WrapSymmetricKey(CCB_HandleTypeDef *hccb,
const uint32_t *pClearAESKey,
CCB_WrappingKeyTypeDef *pWrappingKey);

typedef struct
{
    uint32_t WrappingKeyType;          /*!< This parameter can be value of
                                       @ref CCB_WrappingKeyTypeDef */
    uint32_t AES_Algorithm;           /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, AES Algorithm.
                                       This parameter can be a value of
                                       @ref CCB_AES_Algorithm_Mode */
    uint32_t *pInitVect;              /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, Pointer to
                                       the initialization vector, counter with
                                       CBC Algo */
    uint32_t *pUserWrappedWrappingKey; /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, Pointer to the
                                       wrapped wrapping Key */
} CCB_WrappingKeyTypeDef;

```

Wrap the `ClearPrivateKey` with the `HAL_CCB` function below. Keep the `WrappedPrivateKeyBlob` result in volatile memory.

```

HAL_StatusTypeDef HAL_CCB_ECDSA_WrapPrivateKey(CCB_HandleTypeDef *hccb,
CCB_ECDSACurveParamTypeDef *pCurveParam,
uint8_t *pClearPrivateKey,
CCB_WrappingKeyTypeDef *pWrappingKey,
const CCB_ECDSAKeyBlobTypeDef *pWrappedPrivateKeyBlob);

```

Above function also do the following:

- Generate the public key, as defined in Step 3 (local/ internal variable).
- Generate the initial ECDSA signature, as defined in Step 2 (local/ internal variable).

Lastly, as defined in Step 4, `HAL_CCB_ECDSA_WrapPrivateKey` verifies the generated signature with the generated public key. If the signature verification returns true `WrappedPrivateKeyBlob` can be saved anywhere in non-volatile memory.

Step 5 code:

At any time, a signature can be computed using `WrappedPrivateKeyBlob` and the `HAL_CCB` function:

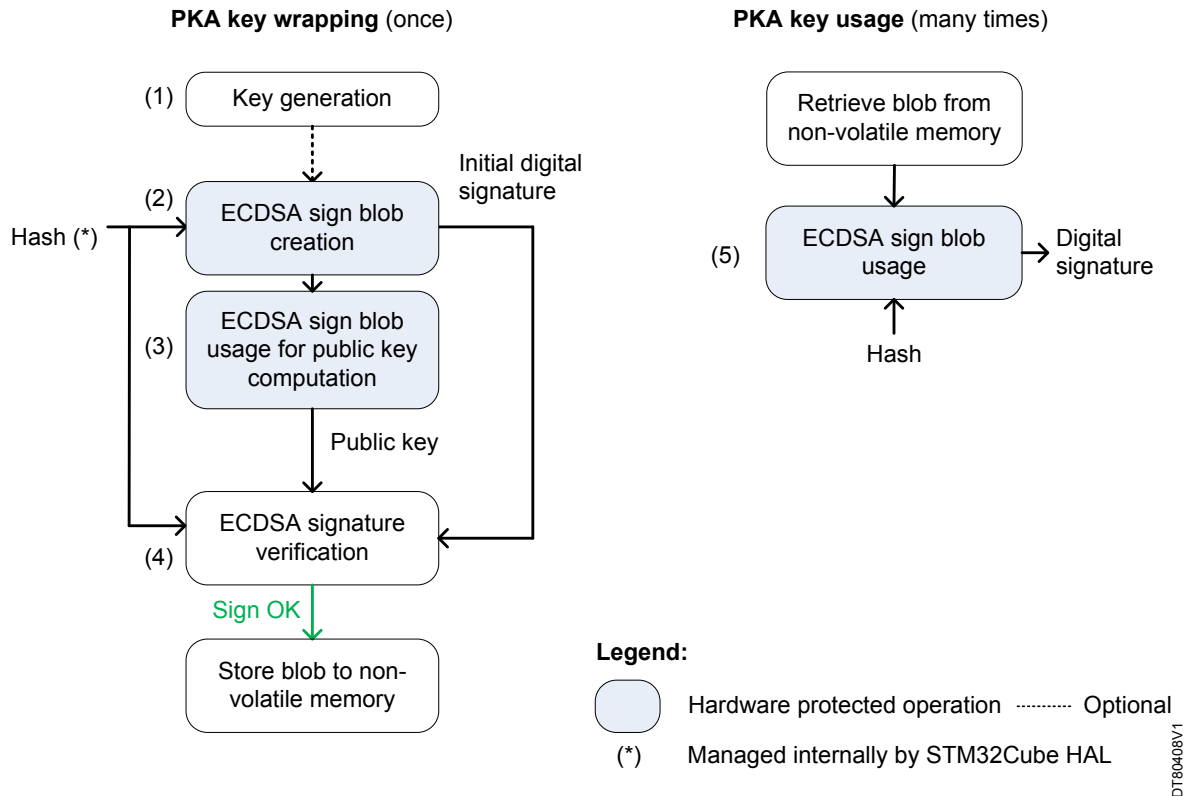
```

HAL_StatusTypeDef HAL_CCB_ECDSA_Sign(CCB_HandleTypeDef *hccb,
CCB_ECDSACurveParamTypeDef *pCurveParam,
CCB_WrappingKeyTypeDef *pWrappingKey,
CCB_ECDSAKeyBlobTypeDef *pWrappedPrivateKeyBlob,
uint8_t *pHash,
CCB_ECDSASignTypeDef *pSignature);

```

3.1.5 Blob creation, blob usage (PT3 code)

Figure 5. ECDSA signature blob creation and usage (PT3)



STM32Cube HAL supports the following methods to implement the flow in Figure 5. These methods apply to product type 3 defined in Table 2:

1. Generate private key (optional).
2. Protected ECDSA signature blob creation, with a selected ECC curve and any hash information. Use the step 1 private key as input, or allow the device to generate a key in a confidential way. The result includes the initial signature (r, s).
3. Protected scalar multiplication, with ECC curve base point G and previously computed blob as input. Result is the computed ECDSA signature public key.
4. ECDSA signature verification uses the reference hash information, the initial signature (r, s), and the computed ECDSA signature public key. If the API returns that the signature is correct, save the ECDSA signature blob created in step 2.
5. ECDSA key blob can be used at any time to compute ECDSA signature using a wrapped key.

Step 1 code (optional):

If the key is generated by application, refer to the Step 1 code in Section 3.1.3.

Step 2, 3 and 4 code:

Wrap the private key with the HAL_CCB function below. Save the reference ECDSA signature (RSign, SSign) and the out_wrapped_private_key_blob in volatile memory.

Use wrapping_hw_key_type to select 256-bit DHUK or DHUK XOR BHK as hardware wrapping key.

- Case 1: Private key `clear_private_key` is generated in Step 1, and wrapping key is hardware.

```
hal_status_t HAL_CCB_ECDSA_HW_WrapPrivateKey(hal_ccb_handle_t *hccb,
const hal_ccb_ecdsa_curve_param_t *p_in_curve_param,
const uint8_t *p_in_clear_private_key,
hal_ccb_wrapping_hw_key_type_t wrapping_hw_key_type,
hal_ccb_ecdsa_key_blob_t *p_out_wrapped_private_key_blob);
```

- Case 2: Private key is generated automatically by the TRNG, and wrapping key is hardware.

```
hal_status_t HAL_CCB_ECDSA_HW_GenerateWrapPrivateKey(hal_ccb_handle_t *hccb,
const hal_ccb_ecdsa_curve_param_t *p_in_curve_param,
hal_ccb_wrapping_hw_key_type_t wrapping_hw_key_type,
hal_ccb_ecdsa_key_blob_t *p_out_wrapped_private_key_blob);
```

If the user wants to wrap the private key with a known `user_key` this user-defined wrapping key must first be encrypted by calling the `HAL_CCB` function below. The recommended algorithm, defined in `wrapping_key_context`, is AES-CBC with 256-bit DHUK.

```
hal_status_t HAL_CCB_ECDSA_WrapSymmetricKey(hal_ccb_handle_t *hccb,
const uint32_t *p_in_user_key,
hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
uint32_t *p_out_wrapped_user_key);
```

Afterward, wrap the private key with the `HAL_CCB` function below. Save the reference ECDSA signature (`RSign`, `SSign`) and `out_wrapped_private_key_blob` in volatile memory.

- Case 1: Private key `clear_private_key` is generated in Step 1, and wrapping key is user-defined (previously computed `out_wrapped_user_key`).

```
hal_status_t HAL_CCB_ECDSA_SW_WrapPrivateKey(hal_ccb_handle_t *hccb,
const hal_ccb_ecdsa_curve_param_t *p_in_curve_param,
const uint8_t *p_in_clear_private_key,
hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
uint32_t *p_in_wrapped_user_key,
hal_ccb_ecdsa_key_blob_t *p_out_wrapped_private_key_blob);
```

- Case 2: Private key is generated automatically by the TRNG, and wrapping key is user-defined (previously computed `out_wrapped_user_key`).

```
hal_status_t HAL_CCB_ECDSA_SW_GenerateWrapPrivateKey(hal_ccb_handle_t *hccb,
const hal_ccb_ecdsa_curve_param_t *p_in_curve_param,
hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
uint32_t *p_in_wrapped_user_key,
hal_ccb_ecdsa_key_blob_t *p_out_wrapped_private_key_blob);
```

All the `WrapPrivateKey()` functions generate the public key (as defined in Step 3), then verify the initial signature using the computed public key, as defined in Step 4. If the signature verification returns true, `wrapped_private_key_blob` can be saved anywhere in non-volatile memory.

Step 5 code:

At any time, a signature can be computed using `wrapped_private_key_blob` and the `HAL_CCB` function:

- Case 1: Wrapping key is hardware. Use `wrapping_hw_key_type` to select 256-bit DHUK or DHUK XOR BHK as hardware wrapping key.

```
hal_status_t HAL_CCB_ECDSA_HW_Sign(hal_ccb_handle_t *hccb,
    const hal_ccb_ecdsa_curve_param_t *p_in_curve_param,
    hal_ccb_wrapping_hw_key_type_t wrapping_hw_key_type,
    hal_ccb_ecdsa_key_blob_t *p_in_wrapped_private_key_blob,
    const uint8_t *p_in_hash,
    uint8_t hash_size,
    hal_ccb_ecdsa_sign_t *p_out_signature);
```

- Case 2: Wrapping key is user-defined (previously computed `wrapped_user_key`).

```
hal_status_t HAL_CCB_ECDSA_SW_Sign(hal_ccb_handle_t *hccb,
    const hal_ccb_ecdsa_curve_param_t *p_in_curve_param,
    hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
    uint32_t *p_in_wrapped_user_key,
    hal_ccb_ecdsa_key_blob_t *p_in_wrapped_private_key_blob,
    const uint8_t *p_in_hash,
    uint8_t hash_size,
    hal_ccb_ecdsa_sign_t *p_out_signature);
```

3.2 Scalar multiplication

STM32Cube HAL proposes a set of functions to run scalar multiplication with PKA wrapped keys.

3.2.1 Key generation

ECC private key is generated with the RNG. In FIPS PUB 186-4 specification “*Digital Signature Standard (DSS)*” section B.4 NIST proposes two methods for the generation of the ECC private key (*extra random bits* or *testing candidates*). Application must select one of those two methods when computing ECC private keys.

Public key computation is described in Step 4 of [Section 3.2.3](#) for product type 1 (PT1), Step 4 of [Section 3.2.4](#) for product type 2 (PT2), and in Step 3 of [Section 3.2.5](#) for product type 3 (PT3).

3.2.2 Blob creation, blob usage

Wrapping an ECC private key corresponds to creating an ECC key blob. Its usage is summarized in [Table 4](#).

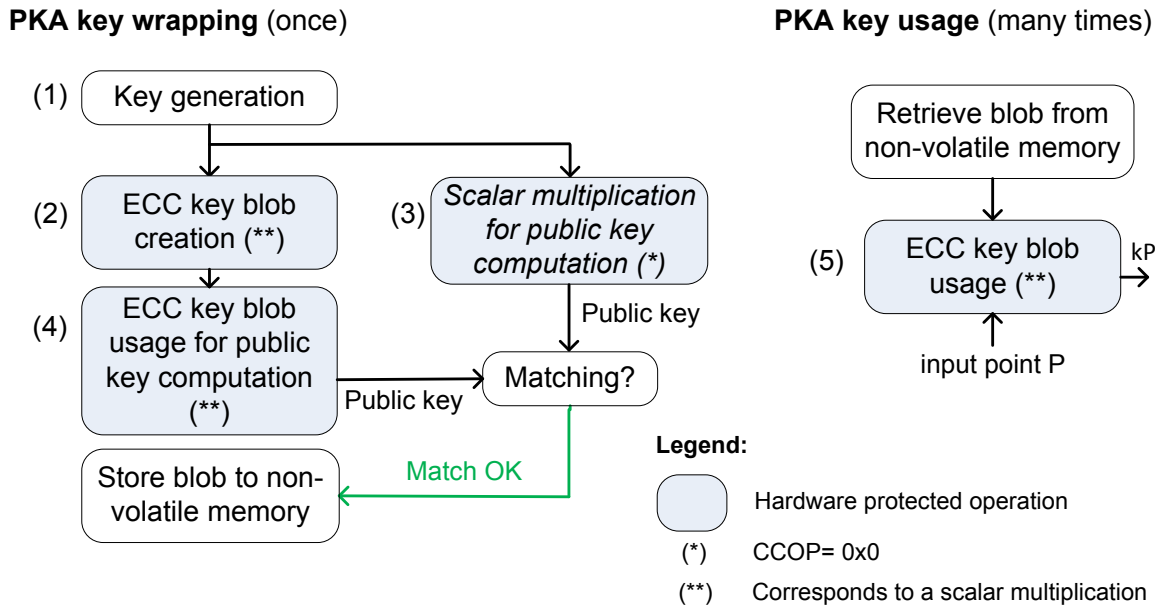
Table 4. ECC key blob usage

Attestation	Key import	Key agreement	Key generation	Usage in PKA
-	X	X	X	Scalar multiplication

ECC key blob creation and usage methods are summarized in [Section 3.2.3](#) for product type 1 (PT1), in [Section 3.2.4](#) for product type 2 (PT2), and in [Section 3.2.5](#) for product type 3 (PT3).

3.2.3 Blob creation, blob usage (PT1 code)

Figure 6. ECC key blob creation and usage (PT1)



STM32Cube HAL supports the following methods to implement the flow in Figure 6. These methods apply to product type 1 defined in Table 2:

1. Generate keys.
2. Protected ECC key blob creation, with selected ECC curve and the generated private key as input.
3. Protected scalar multiplication, with ECC curve base point G and previously generated private key as input. The result is the reference public key.
4. Protected scalar multiplication, with ECC curve base point G and previously computed ECC key blob as input. The result is the computed public key. If reference and computed points are identical, save the ECC key blob created in step 2.
5. ECC key blob can be used at any time to compute scalar multiplication using a wrapped key.

Step 1 code:

Generate private key `ClearPrivateKey` with the TRNG, as defined in Section 3.2.1.

Step 2, 3 and 4 code:

If the user wants to wrap `ClearPrivateKey` with DHUK (or DHUK XOR BHK) select

`WrappingKeyType = HAL_CCB_USER_KEY_HW (or HAL_CCB_USER_KEY_HSW).`

If the user wants to wrap `ClearPrivateKey` with a known `ClearAESKey` this user-defined wrapping key must

first be encrypted by calling the `HAL_CCB` function below, with `WrappingKeyType =`

`HAL_CCB_USER_KEY_WRAPPED.` The recommended algorithm is AES-CBC with 256-bit DHUK, selecting

`AES_Algorithm = CCB_AES_CBC` and initializing `InitVect`.

```

HAL_StatusTypeDef HAL_CCB_ECC_WrapSymmetricKey(CCB_HandleTypeDef *hccb,
const uint32_t *pClearAESKey,
CCB_WrappingKeyTypeDef *pWrappingKey);

```

```
typedef struct
{
    uint32_t WrappingKeyType;          /*!< This parameter can be value of
                                       @ref CCB_WrappingKeyTypeDef          */
    uint32_t AES_Algorithm;            /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, AES Algorithm.
                                       This parameter can be a value of
                                       @ref CCB_AES_Algorithm_Mode          */
    uint32_t *pInitVect;              /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, Pointer to
                                       the initialization vector,
                                       counter with CBC Algo                      */
    uint32_t *pUserWrappedWrappingKey; /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, Pointer to
                                       the wrapped wrapping Key                      */
} CCB_WrappingKeyTypeDef;
```

Wrap the `ClearPrivateKey` with the `HAL_CCB` function below. Keep the `WrappedPrivateKeyBlob` result in volatile memory.

```
HAL_StatusTypeDef HAL_CCB_ECC_WrapPrivateKey(CCB_HandleTypeDef *hccb,
    CCB_ECCMulCurveParamTypeDef *pCurveParam,
    uint8_t *pClearPrivateKey,
    CCB_WrappingKeyTypeDef *pWrappingKey,
    CCB_ECCMulKeyBlobTypeDef *pWrappedPrivateKeyBlob);
```

Above function also do the following:

- Compute the reference public key, as defined in Step 3 (local/ internal variable).
- Generate the public key, as defined in Step 4 (local/ internal variable).

Lastly, `HAL_CCB_ECC_WrapPrivateKey()` compares both public keys. If they match `WrappedPrivateKeyBlob` can be saved anywhere in non-volatile memory.

Step 5 code:

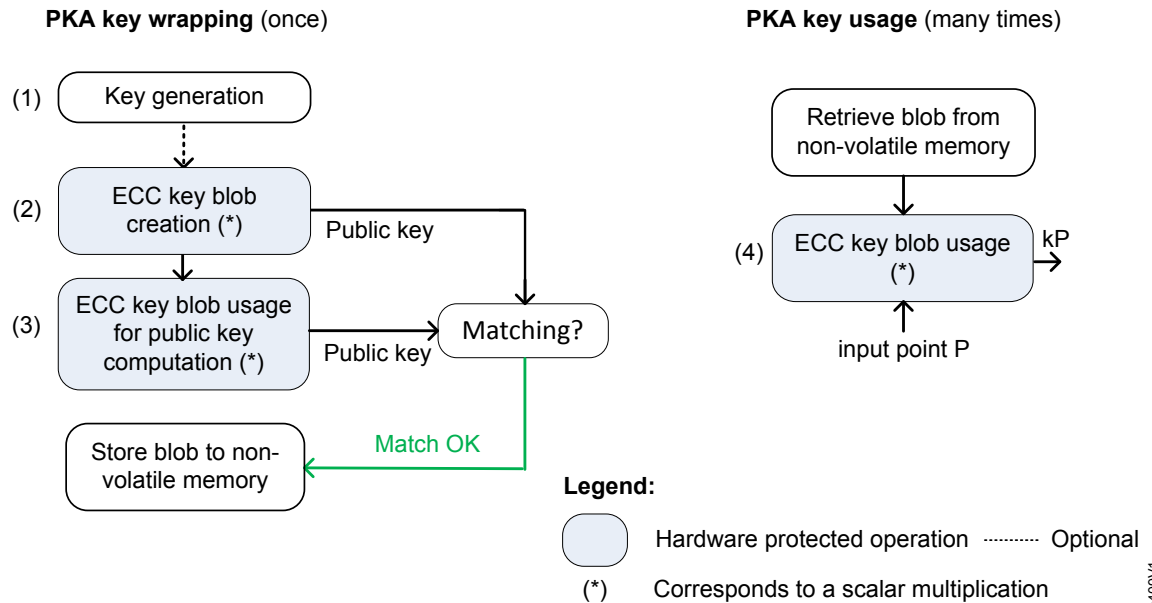
At any time, a scalar multiplication can be computed using `WrappedPrivateKeyBlob` and the `HAL_CCB` function below.

```
HAL_StatusTypeDef HAL_CCB_ECC_ComputeScalarMul(CCB_HandleTypeDef *hccb,
    CCB_ECCMulCurveParamTypeDef *pCurveParam,
    CCB_WrappingKeyTypeDef *pWrappingKey,
    CCB_ECCMulKeyBlobTypeDef *pWrappedPrivateKeyBlob,
    CCB_ECCMulPointTypeDef *pInputPoint,
    CCB_ECCMulPointTypeDef *pOutputPoint);
```

```
typedef struct
{
    uint8_t *pPointX;          /*!< Pointer to point P coordinate xP */
    uint8_t *pPointY;          /*!< Pointer to point P coordinate yP */
} CCB_ECCMulPointTypeDef;
```

3.2.4 Blob creation, blob usage (PT2 code)

Figure 7. ECC key blob creation and usage (PT2)



STM32Cube HAL supports the following methods to implement the flow on Figure 7. These methods apply to product type 2 defined in Table 2:

1. Generate private key (optional).
2. Protected ECC key blob creation, with a selected ECC curve. Use the step 1 private key as input, or allow the device to generate a key in a confidential way. The result includes the reference public key.
3. Protected scalar multiplication, with ECC curve base point G and previously computed ECC key blob as input. The result is the computed public key. If reference and computed public keys are identical, save the ECC key blob created in step 2.
4. ECC key blob can be used at any time to compute scalar multiplication using a wrapped key.

Step 1 code (optional):

If the key is generated by application, refer to the Step 1 code in Section 3.2.3.

Step 2 and 3 code:

If the user wants to wrap `ClearPrivateKey` with DHUK (or DHUK XOR BHK) select `WrappingKeyType = HAL_CCB_USER_KEY_HW` (or `HAL_CCB_USER_KEY_HSW`).

If the user wants to wrap `ClearPrivateKey` with a known `ClearAESKey` this user-defined wrapping key must first be encrypted by calling the `HAL_CCB` function below, with `WrappingKeyType = HAL_CCB_USER_KEY_WRAPPED`. The recommended algorithm is AES-CBC with 256-bit DHUK, selecting `AES_Algorithm = CCB_AES_CBC` and initializing `InitVect`.

```

HAL_StatusTypeDef HAL_CCB_ECC_WrapSymmetricKey(CCB_HandleTypeDef *hccb,
const uint32_t *pClearAESKey,
CCB_WrappingKeyTypeDef *pWrappingKey);

```

```
typedef struct
{
    uint32_t WrappingKeyType;          /*!< This parameter can be value of
                                       @ref CCB_WrappingKeyTypeDef          */
    uint32_t AES_Algorithm;            /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, AES Algorithm.
                                       This parameter can be a value of
                                       @ref CCB_AES_Algorithm_Mode          */
    uint32_t *pInitVect;              /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, Pointer to
                                       the initialization vector,
                                       counter with CBC Algo                      */
    uint32_t *pUserWrappedWrappingKey; /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, Pointer to
                                       the wrapped wrapping Key                      */
} CCB_WrappingKeyTypeDef;
```

Wrap the `ClearPrivateKey` with the `HAL_CCB` function below. Keep the `WrappedPrivateKeyBlob` result in volatile memory.

```
HAL_StatusTypeDef HAL_CCB_ECC_WrapPrivateKey(CCB_HandleTypeDef *hccb,
    CCB_ECCMulCurveParamTypeDef *pCurveParam,
    uint8_t *pClearPrivateKey,
    CCB_WrappingKeyTypeDef *pWrappingKey,
    CCB_ECCMulKeyBlobTypeDef *pWrappedPrivateKeyBlob);
```

Above function also do the following:

- Compute the reference public key as defined in Step 2 (local/ internal variable).
- Generate the public key as defined in Step 3 (local/ internal variable).

Lastly, `HAL_CCB_ECC_WrapPrivateKey()` compares both public keys. If they match `WrappedPrivateKeyBlob` can be saved anywhere in non-volatile memory.

Step 4 code:

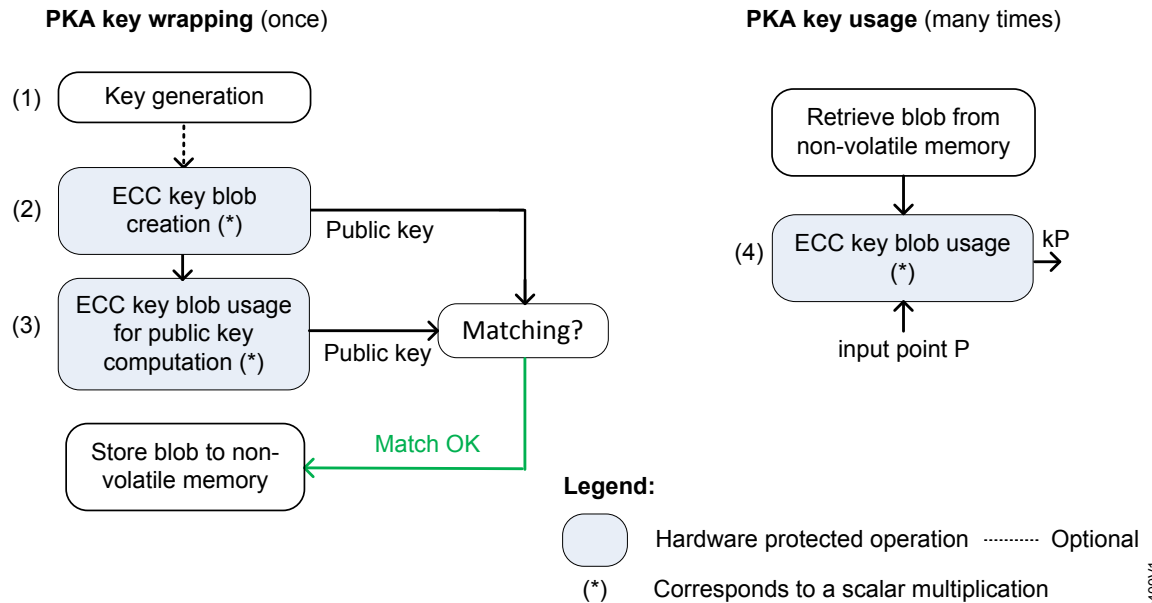
At any time, a scalar multiplication can be computed using `WrappedPrivateKeyBlob` and the `HAL_CCB` function below.

```
HAL_StatusTypeDef HAL_CCB_ECC_ComputeScalarMul(CCB_HandleTypeDef *hccb,
    CCB_ECCMulCurveParamTypeDef *pCurveParam,
    CCB_WrappingKeyTypeDef *pWrappingKey,
    CCB_ECCMulKeyBlobTypeDef *pWrappedPrivateKeyBlob,
    CCB_ECCMulPointTypeDef *pInputPoint,
    CCB_ECCMulPointTypeDef *pOutputPoint);
```

```
typedef struct
{
    uint8_t *pPointX;          /*!< Pointer to point P coordinate xP */
    uint8_t *pPointY;          /*!< Pointer to point P coordinate yP */
} CCB_ECCMulPointTypeDef;
```

3.2.5 Blob creation, blob usage (PT3 code)

Figure 8. ECC key blob creation and usage (PT3)



STM32Cube HAL supports the following methods to implement the flow on Figure 8. These methods apply to product type 3 defined in Table 2:

1. Generate private key (optional).
2. Protected ECC key blob creation, with a selected ECC curve. Use the step 1 private key as input, or allow the device to generate a key in a confidential way. The result includes the reference public key.
3. Protected scalar multiplication, with ECC curve base point G and previously computed ECC key blob as input. The result is the computed public key. If reference and computed public keys are identical, save the ECC key blob created in step 2.
4. ECC key blob can be used at any time to compute scalar multiplication using a wrapped key.

Step 1 code (optional):

If the key is generated by application, refer to the Step 1 code in Section 3.2.3.

Step 2 and 3 code:

Wrap the private key with the HAL_CCB function below. Save the reference public key and the `out_wrapped_private_key_blob` results in volatile memory. Use `wrapping_hw_key_type` to select 256-bit DHUK or DHUK XOR BHK as hardware wrapping key. ECC curve base point G must be written as input (`pointX`, `pointY`).

- Case 1: Private key `clear_private_key` is generated in Step 1, and wrapping key is hardware.

```

hal_status_t HAL_CCB_ECC_HW_WrapPrivateKey(hal_ccb_handle_t *hccb,
const hal_ccb_ecc_mul_curve_param_t *p_in_curve_param,
const uint8_t *p_in_clear_private_key,
hal_ccb_wrapping_hw_key_type_t wrapping_hw_key_type,
hal_ccb_ecc_mul_key_blob_t *p_out_wrapped_private_key_blob);

```

- Case 2: Private key is generated automatically by the TRNG, and wrapping key is hardware.

```

hal_status_t HAL_CCB_ECC_HW_GenerateWrapPrivateKey(hal_ccb_handle_t *hccb,
const hal_ccb_ecc_mul_curve_param_t *p_in_curve_param,
hal_ccb_wrapping_hw_key_type_t wrapping_hw_key_type,
hal_ccb_ecc_mul_key_blob_t *p_out_wrapped_private_key_blob);

```

If the user wants to wrap the private key with a known `user_key`, this user-defined wrapping key must first be encrypted by calling the `HAL_CCB` function below. The recommended algorithm, defined in `wrapping_key_context`, is AES-CBC with 256-bit DHUK.

```
hal_status_t HAL_CCB_ECC_WrapSymmetricKey(hal_ccb_handle_t *hccb,
    const uint32_t *p_in_user_key,
    hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
    uint32_t *p_out_wrapped_user_key);
```

Afterward, wrap the private key with the `HAL_CCB` function below. Save the reference public key and the `out_wrapped_private_key_blob` results in volatile memory.

- Case 1: Private key `clear_private_key` is generated in Step 1, and wrapping key is user-defined (previously computed `out_wrapped_user_key`).

```
hal_status_t HAL_CCB_ECC_SW_WrapPrivateKey(hal_ccb_handle_t *hccb,
    const hal_ccb_ecc_mul_curve_param_t *p_in_curve_param,
    const uint8_t *p_in_clear_private_key,
    hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
    uint32_t *p_in_wrapped_user_key,
    hal_ccb_ecc_mul_key_blob_t *p_out_wrapped_private_key_blob);
```

- Case 2: Private key is generated automatically by the TRNG, and wrapping key is user-defined (previously computed `out_wrapped_user_key`).

```
hal_status_t HAL_CCB_ECC_SW_GenerateWrapPrivateKey(hal_ccb_handle_t *hccb,
    const hal_ccb_ecc_mul_curve_param_t *p_in_curve_param,
    hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
    uint32_t *p_in_wrapped_user_key,
    hal_ccb_ecc_mul_key_blob_t *p_out_wrapped_private_key_blob);
```

All the `WrapPrivateKey()` functions generate the public key (as defined in Step 3), then verify it against the reference public key that was saved as `local/` internal variable. If the values match, `wrapped_private_key_blob` can be saved anywhere in non-volatile memory.

Step 4 code:

At any time, a scalar multiplication can be computed using this `HAL_CCB` function.

- Case 1: Wrapping key is hardware. Use `wrapping_hw_key_type` to select 256-bit DHUK or DHUK XOR BHK as hardware wrapping key.

```
hal_status_t HAL_CCB_ECC_HW_ComputeScalarMul(hal_ccb_handle_t *hccb,
    const hal_ccb_ecc_mul_curve_param_t *p_in_curve_param,
    hal_ccb_wrapping_hw_key_type_t wrapping_hw_key_type,
    hal_ccb_ecc_mul_key_blob_t *p_in_wrapped_private_key_blob,
    hal_ccb_ecc_point_t *p_in_point,
    hal_ccb_ecc_point_t *p_out_point);
```

- Case 2: Wrapping key is user-defined (previously computed `wrapped_user_key`).

```
hal_status_t HAL_CCB_ECC_SW_ComputeScalarMul(hal_ccb_handle_t *hccb,
    const hal_ccb_ecc_mul_curve_param_t *p_in_curve_param,
    hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
    uint32_t *p_in_wrapped_user_key,
    hal_ccb_ecc_mul_key_blob_t *p_in_wrapped_private_key_blob,
    hal_ccb_ecc_point_t *p_in_point,
    hal_ccb_ecc_point_t *p_out_point);
```

3.3 Modular exponentiation

STM32Cube HAL proposes a set of functions to compute modular exponentiation with PKA wrapped keys.

3.3.1 Key generation

RSA key generation is out of scope of this document.

3.3.2 Blob creation, blob usage

Wrapping an RSA private key corresponds to creating an RSA key blob. Its usage is summarized in [Table 5](#).

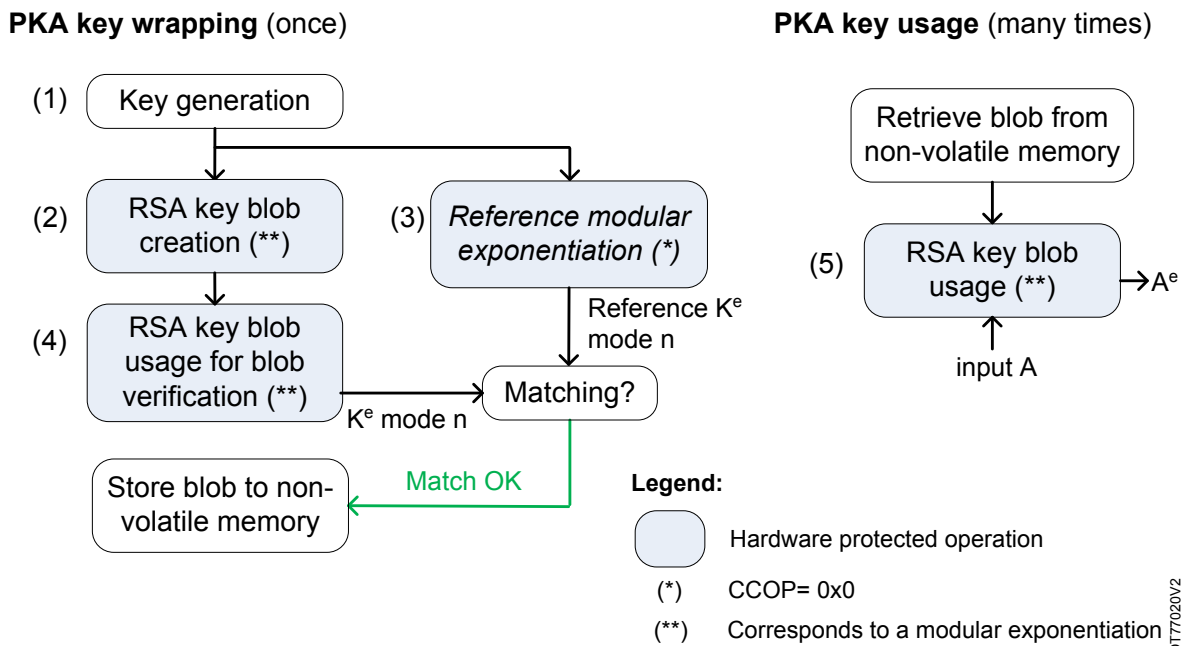
Table 5. RSA key blob usage

Attestation	Key import	Key agreement	Key generation	Usage in PKA
X	X	X	-	Modular exponentiation

RSA key blob creation and usage methods are summarized in Section 3.3.3 for product type 1 (PT1), in Section 3.3.4 for product type 2 (PT2), and in Section 3.3.5 for product type 3 (PT3).

3.3.3 Blob creation, blob usage (PT1 code)

Figure 9. RSA key blob creation and usage (PT1)



STM32Cube HAL supports the following methods to implement the flow in Figure 9. These methods apply to product type 1 defined in Table 2:

1. Generate keys.
2. Protected RSA key blob creation, with selected RSA parameters and the generated private key as input.
3. Protected modular exponentiation, using a fixed value K and a previously generated private key as input. The result is the reference $K^e \text{ mode } n$.
4. Protected modular exponentiation, using the fixed value K of step 3 and the previously computed RSA key blob as input. The result is the computed $K^e \text{ mode } n$. If reference and computed values are identical, save the RSA key blob created in step 2.
5. RSA key blob can be used at any time to compute modular exponentiation using a wrapped key.

Step 1 code:

As the key is not generated on the microcontroller (see Section 3.3.1) this part of the code retrieves the RSA key components from local key storage.

Step 2, 3 and 4 code:

If the user wants to wrap `RSAClearPrivateKey` with DHUK (or DHUK XOR BHK) select `WrappingKeyType = HAL_CCB_USER_KEY_HW` (or `HAL_CCB_USER_KEY_HSW`).

If the user wants to wrap `RSAClearPrivateKey` with a known `ClearAESKey` this user-defined wrapping key must first be encrypted by calling the `HAL_CCB` function below, with `WrappingKeyType = HAL_CCB_USER_KEY_WRAPPED`. The recommended algorithm is AES-CBC with 256-bit DHUK, selecting `AES_Algorithm = CCB_AES_CBC` and initializing `InitVect`.

```
HAL_StatusTypeDef HAL_CCB_RSA_WrapSymmetricKey(CCB_HandleTypeDef *hccb,
const uint32_t *pClearAESKey,
CCB_WrappingKeyTypeDef *pWrappingKey);

typedef struct
{
    uint32_t WrappingKeyType;          /*!< This parameter can be value of
    @ref CCB_WrappingKeyTypeDef */
    uint32_t AES_Algorithm;            /*!< Used only when wrappingkey type is
    HAL_CCB_USER_KEY_WRAPPED, AES
    Algorithm. This parameter can be a
    value of @ref CCB_AES_Algorithm_Mode */
    uint32_t *pInitVect;              /*!< Used only when wrappingkey type is
    HAL_CCB_USER_KEY_WRAPPED, Pointer
    to the initialization vector,
    counter with CBC Algo */
    uint32_t *pUserWrappedWrappingKey; /*!< Used only when wrappingkey type is
    HAL_CCB_USER_KEY_WRAPPED, Pointer
    to the wrapped wrapping Key */
} CCB_WrappingKeyTypeDef;
```

Wrap the `RSAClearPrivateKey` with the `HAL_CCB` function below. Keep the `WrappedPrivateKeyBlob` result in volatile memory.

```
HAL_StatusTypeDef HAL_CCB_RSA_WrapPrivateKey(CCB_HandleTypeDef *hccb,
CCB_RSAParamTypeDef *pParam,
CCB_RSAClearKeyTypeDef *pRSAClearPrivateKey,
CCB_WrappingKeyTypeDef *pWrappingKey,
CCB_RSAKeyBlobTypeDef *pWrappedPrivateKeyBlob);
```

Above function also do the following:

- Generate the reference $K^e \text{ mode } n$, as defined in Step 2 (local/ internal variable).
- Generate $K^e \text{ mode } n$ as defined in Step 3 (local/ internal variable).

Lastly, `HAL_CCB_RSA_WrapPrivateKey()` compares both $K^e \text{ mode } n$. If they match `WrappedPrivateKeyBlob` can be saved anywhere in non-volatile memory.

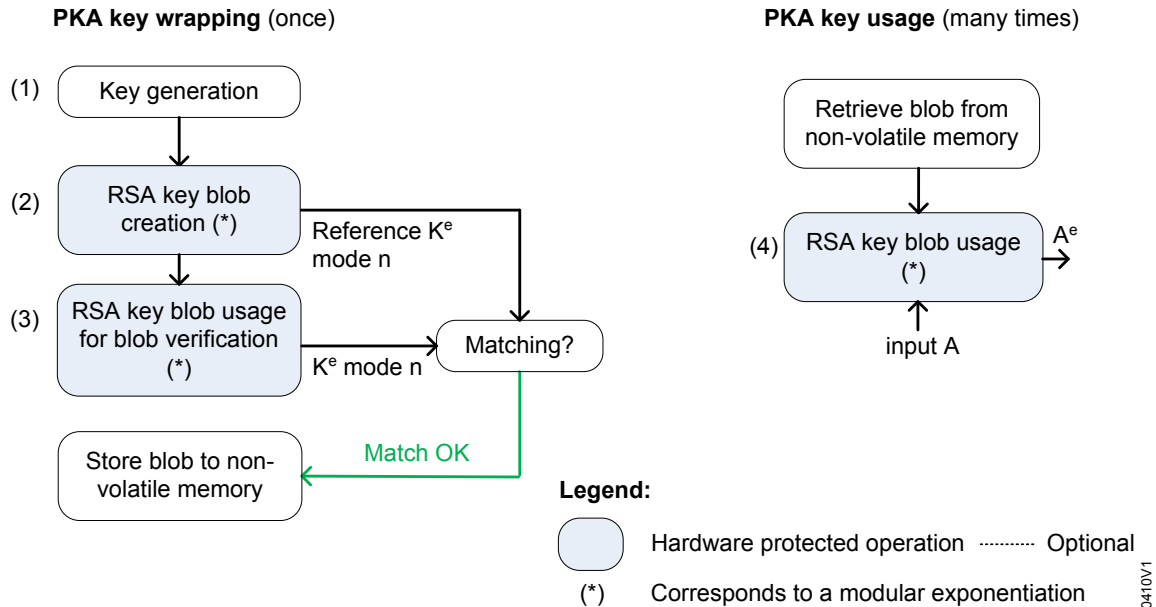
Step 5 code:

At any time, a modular exponentiation can be computed using `WrappedPrivateKeyBlob` and the `HAL_CCB` function.

```
HAL_StatusTypeDef HAL_CCB_RSA_ComputeModularExp(CCB_HandleTypeDef *hccb,
CCB_RSAParamTypeDef *pParam,
CCB_WrappingKeyTypeDef *pWrappingKey,
CCB_RSAKeyBlobTypeDef *pWrappedPrivateKeyBlob,
uint8_t *pOperand,
uint8_t *pModularExp);
```

3.3.4 Blob creation, blob usage (PT2 code)

Figure 10. RSA key blob creation and usage (PT2)



STM32Cube HAL supports the following methods to implement the flow on Figure 10. These methods apply to product type 2 defined in Table 2:

1. Generate keys.
2. Protected RSA key blob creation, with selected RSA parameters, a fixed value K, and the generated private key as input. The result includes the reference $K^e \text{ mode } n$.
3. Protected modular exponentiation, using the previously computed RSA key blob as input. Also use the fixed value K of Step 2. The result is the computed $K^e \text{ mode } n$. If reference and computed values are identical, save the RSA key blob created in step 2.
4. RSA key blob can be used at any time to compute modular exponentiation using a wrapped key.

Step 1 code:

As the key is not generated on the microcontroller (see Section 3.3.1) this part of the code retrieves the RSA key components from local key storage.

Step 2 and 3 code:

If the user wants to wrap `RSAClearPrivateKey` with DHUK (or DHUK XOR BHK) select `WrappingKeyType = HAL_CCB_USER_KEY_HW` (or `HAL_CCB_USER_KEY_HSW`).

If the user wants to wrap `RSAClearPrivateKey` with a known `ClearAESKey` this user-defined wrapping key must first be encrypted by calling the `HAL_CCB` function below, with

`WrappingKeyType = HAL_CCB_USER_KEY_WRAPPED`. The recommended algorithm is AES-CBC with 256-bit DHUK, selecting `AES_Algorithm = CCB_AES_CBC` and initializing `InitVect`.

```

HAL_StatusTypeDef HAL_CCB_RSA_WrapSymmetricKey(CCB_HandleTypeDef *hccb,
const uint32_t *pClearAESKey,
CCB_WrappingKeyTypeDef *pWrappingKey);

```

```
typedef struct
{
    uint32_t WrappingKeyType;          /*!< This parameter can be value of
                                       @ref CCB_WrappingKeyTypeDef          */
    uint32_t AES_Algorithm;            /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, AES
                                       Algorithm. This parameter can be a
                                       value of @ref CCB_AES_Algorithm_Mode */
    uint32_t *pInitVect;               /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, Pointer
                                       to the initialization vector,
                                       counter with CBC Algo                */
    uint32_t *pUserWrappedWrappingKey; /*!< Used only when wrappingkey type is
                                       HAL_CCB_USER_KEY_WRAPPED, Pointer
                                       to the wrapped wrapping Key          */
} CCB_WrappingKeyTypeDef;
```

Wrap the RSAClearPrivateKey with the HAL_CCB function below. Keep the WrappedPrivateKeyBlob result in volatile memory.

```
HAL_StatusTypeDef HAL_CCB_RSA_WrapPrivateKey(CCB_HandleTypeDef *hccb,
    CCB_RSAParamTypeDef *pParam,
    CCB_RSAClearKeyTypeDef *pRSAClearPrivateKey,
    CCB_WrappingKeyTypeDef *pWrappingKey,
    CCB_RSAKeyBlobTypeDef *pWrappedPrivateKeyBlob);
```

Above function also do the following:

- Generate the reference $K^e \text{ mode } n$, as defined in Step 3 (local/ internal variable).
- Generate $K^e \text{ mode } n$ as defined in Step 4 (local/ internal variable).

Lastly, HAL_CCB_RSA_WrapPrivateKey() compares both $K^e \text{ mode } n$. If they match WrappedPrivateKeyBlob can be saved anywhere in non-volatile memory.

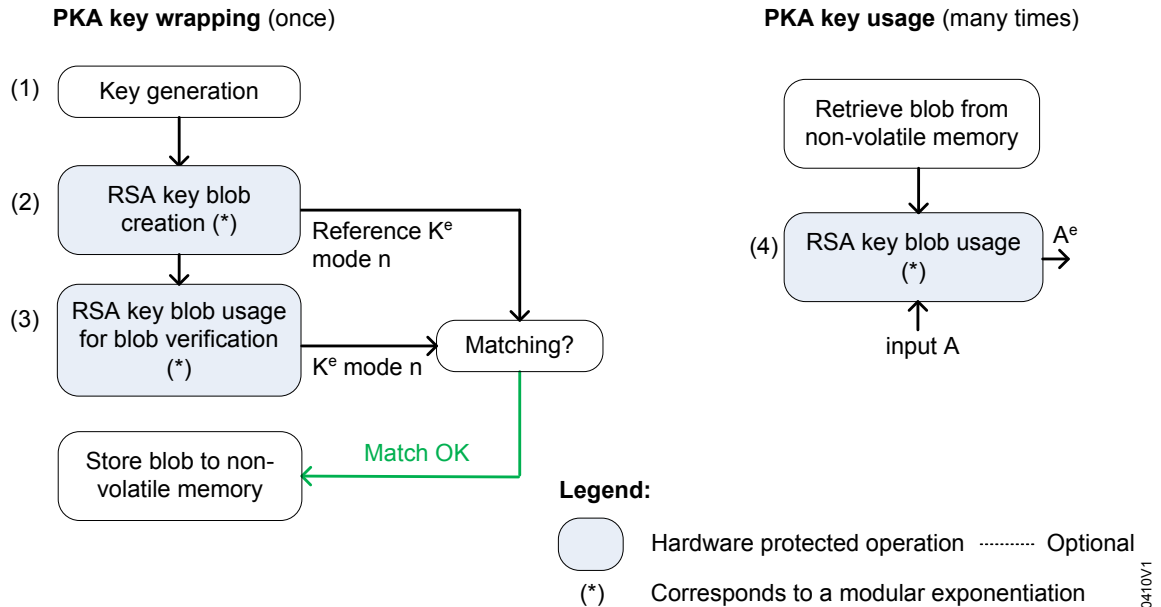
Step 4 code:

At any time, a modular exponentiation can be computed using WrappedPrivateKeyBlob and the HAL_CCB function below.

```
HAL_StatusTypeDef HAL_CCB_RSA_ComputeModularExp(CCB_HandleTypeDef *hccb,
    CCB_RSAParamTypeDef *pParam,
    CCB_WrappingKeyTypeDef *pWrappingKey,
    CCB_RSAKeyBlobTypeDef *pWrappedPrivateKeyBlob,
    uint8_t *pOperand,
    uint8_t *pModularExp);
```

3.3.5 Blob creation, blob usage (PT3 code)

Figure 11. RSA key blob creation and usage (PT3)



STM32Cube HAL supports the following methods to implement the flow on Figure 11. These methods apply to product type 3 defined in Table 2:

1. Generate keys.
2. Protected RSA key blob creation, with selected RSA parameters, a fixed value K, and the generated private key as input. The result includes the reference $K^e \text{ mode } n$.
3. Protected modular exponentiation, using the previously computed RSA key blob as input. Also use the fixed value K of Step 2. The result is the computed $K^e \text{ mode } n$. If reference and computed values are identical, save the RSA key blob created in step 2.
4. RSA key blob can be used at any time to compute modular exponentiation using a wrapped key.

Step 1 code:

As the key is not generated on the microcontroller (see Section 3.3.1) this part of the code retrieves the RSA key components from local key storage.

Step 2 and 3 code:

Wrap the private key with the HAL_CCB function below. Save the reference $K^e \text{ mode } n$ and the out_wrapped_private_key_blob results in volatile memory. Use wrapping_hw_key_type to select 256-bit DHUK or DHUK XOR BHK as hardware wrapping key. This function manages the mandatory K parameter.

```

hal_status_t HAL_CCB_RSA_HW_WrapPrivateKey(hal_ccb_handle_t *hccb,
const hal_ccb_rsa_param_t *p_in_param,
const hal_ccb_rsa_clear_key_t *p_in_rsa_clear_private_key,
hal_ccb_wrapping_hw_key_type_t wrapping_hw_key_type,
hal_ccb_rsa_key_blob_t *p_out_wrapped_private_key_blob);
  
```

If the user wants to wrap the private key with a known user_key this user-defined wrapping key must first be encrypted by calling the HAL_CCB function below. The recommended algorithm, defined in wrapping_key_context, is AES-CBC with 256-bit DHUK.

```

hal_status_t HAL_CCB_RSA_WrapSymmetricKey(hal_ccb_handle_t *hccb,
const uint32_t *p_in_user_key,
hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
uint32_t *p_out_wrapped_user_key);
  
```

Afterward, wrap the private key with the HAL_CCB function below. Save the reference $K^e \text{ mode } n$ and the `out_wrapped_private_key_blob` results in volatile memory.

```
hal_status_t HAL_CCB_RSA_SW_WrapPrivateKey(hal_ccb_handle_t *hccb,
    const hal_ccb_rsa_param_t *p_in_param,
    const hal_ccb_rsa_clear_key_t *p_in_rsa_clear_private_key,
    hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
    uint32_t *p_in_wrapped_user_key,
    hal_ccb_rsa_key_blob_t *p_out_wrapped_private_key_blob);
```

All the WrapPrivateKey() functions generate $K^e \text{ mode } n$ (as defined in Step 3), then verify it against the reference $K^e \text{ mode } n$ that was saved as local/ internal variable. If the values match, `wrapped_private_key_blob` can be saved anywhere in non-volatile memory.

Step 4 code:

At any time, a modular exponentiation can be computed using this HAL_CCB function.

- Case 1: Wrapping key is hardware. Use `wrapping_hw_key_type` to select 256-bit DHUK or DHUK XOR BHK as hardware wrapping key.

```
hal_status_t HAL_CCB_RSA_HW_ComputeModularExp(hal_ccb_handle_t *hccb,
    const hal_ccb_rsa_param_t *p_in_param,
    hal_ccb_wrapping_hw_key_type_t wrapping_hw_key_type,
    hal_ccb_rsa_key_blob_t *p_in_wrapped_private_key_blob,
    const uint8_t *p_out_operand, uint8_t *p_out_modular_exp);
```

- Case 2: Wrapping key is user-defined (previously computed `wrapped_user_key`).

```
hal_status_t HAL_CCB_RSA_SW_ComputeModularExp(hal_ccb_handle_t *hccb,
    const hal_ccb_rsa_param_t *p_in_param,
    hal_ccb_wrapping_sw_key_context_t *p_wrapping_key_context,
    uint32_t *p_in_wrapped_user_key,
    hal_ccb_rsa_key_blob_t *p_in_wrapped_private_key_blob,
    const uint8_t *p_out_operand, uint8_t *p_out_modular_exp);
```

4 Conclusion

In IoT devices, the usage of cryptographic private keys can be critical for device security. These STM32 MCUs offer the PKA key wrapping mechanism to ensure the usability of those private keys without disclosing them in cleartext. STM32Cube HAL has a set of functions that makes this feature easier to use for developers.

Revision history

Table 6. Document revision history

Date	Version	Changes
03-Mar-2025	1	Initial release.
16-Feb-2026	2	Updated: - Section 2.2: AES: Key wrapping in SAES peripheral - Section 2.3: PKA: Scalar multiplication - Section 2.4: PKA: ECDSA signature - Section 3.1.2: Blob creation, blob usage - Section 3.2.2: Blob creation, blob usage - Section 3.3.2: Blob creation, blob usage - Figure 3. ECDSA signature blob creation and usage (PT1) - Figure 6. ECC key blob creation and usage (PT1) - Figure 9. RSA key blob creation and usage (PT1) - Table 1. Applicable products Added: - Section 3.1.3: Blob creation, blob usage (PT1 code) - Section 3.1.4: Blob creation, blob usage (PT2 code) - Section 3.1.5: Blob creation, blob usage (PT3 code) - Section 3.2.3: Blob creation, blob usage (PT1 code) - Section 3.2.4: Blob creation, blob usage (PT2 code) - Section 3.2.5: Blob creation, blob usage (PT3 code) - Section 3.3.3: Blob creation, blob usage (PT1 code) - Section 3.3.4: Blob creation, blob usage (PT2 code) - Section 3.3.5: Blob creation, blob usage (PT3 code) - Figure 4. ECDSA signature blob creation and usage (PT2) - Figure 5. ECDSA signature blob creation and usage (PT3) - Figure 7. ECC key blob creation and usage (PT2) - Figure 8. ECC key blob creation and usage (PT3) - Figure 10. RSA key blob creation and usage (PT2) - Figure 11. RSA key blob creation and usage (PT3) - Table 2. Definition of product types
12-Jun-2026	3	Updated with STM32H553xx devices: - Table 1. Applicable products - Table 2. Definition of product types

Contents

1	Why do we need PKA key wrapping?	2
1.1	Private key usage	2
1.2	Private key protection	2
1.3	Private key wrapping in STM32	3
2	Cryptographic primitives with PKA key wrapping	4
2.1	AES: AES-GCM authenticated encryption with associated data (AEAD)	4
2.2	AES: Key wrapping in SAES peripheral	4
2.3	PKA: Scalar multiplication	4
2.4	PKA: ECDSA signature	5
2.5	PKA: Modular exponentiation	6
3	Wrapped key usage with the CCB	7
3.1	ECDSA signature	7
3.1.1	Key generation	7
3.1.2	Blob creation, blob usage	7
3.1.3	Blob creation, blob usage (PT1 code)	8
3.1.4	Blob creation, blob usage (PT2 code)	10
3.1.5	Blob creation, blob usage (PT3 code)	12
3.2	Scalar multiplication	14
3.2.1	Key generation	14
3.2.2	Blob creation, blob usage	14
3.2.3	Blob creation, blob usage (PT1 code)	15
3.2.4	Blob creation, blob usage (PT2 code)	17
3.2.5	Blob creation, blob usage (PT3 code)	19
3.3	Modular exponentiation	20
3.3.1	Key generation	20
3.3.2	Blob creation, blob usage	20
3.3.3	Blob creation, blob usage (PT1 code)	21
3.3.4	Blob creation, blob usage (PT2 code)	23
3.3.5	Blob creation, blob usage (PT3 code)	25
4	Conclusion	27
	Revision history	28
	List of tables	30
	List of figures	31

List of tables

Table 1.	Applicable products	1
Table 2.	Definition of product types.	7
Table 3.	ECDSA signature key blob usage	7
Table 4.	ECC key blob usage.	14
Table 5.	RSA key blob usage.	21
Table 6.	Document revision history	28

List of figures

Figure 1.	Cryptographic usages for private keys	2
Figure 2.	AES-GCM usage for PKA key wrapping	4
Figure 3.	ECDSA signature blob creation and usage (PT1)	8
Figure 4.	ECDSA signature blob creation and usage (PT2)	10
Figure 5.	ECDSA signature blob creation and usage (PT3)	12
Figure 6.	ECC key blob creation and usage (PT1)	15
Figure 7.	ECC key blob creation and usage (PT2)	17
Figure 8.	ECC key blob creation and usage (PT3)	19
Figure 9.	RSA key blob creation and usage (PT1).	21
Figure 10.	RSA key blob creation and usage (PT2).	23
Figure 11.	RSA key blob creation and usage (PT3).	25

IMPORTANT NOTICE – READ CAREFULLY

STMicroelectronics NV and its subsidiaries ("ST") reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice.

In the event of any conflict between the provisions of this document and the provisions of any contractual arrangement in force between the purchasers and ST, the provisions of such contractual arrangement shall prevail.

The purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST's terms and conditions of sale in place at the time of order acknowledgment.

The purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of the purchasers' products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

If the purchasers identify an ST product that meets their functional and performance requirements but that is not designated for the purchasers' market segment, the purchasers shall contact ST for more information.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2026 STMicroelectronics – All rights reserved